



2025-後学期

令和7年度 プログラミング演習 グループプログラミング レポート

「ネットワーク対戦型 2Dシューティングゲーム」

プログラム	メディア情報学
クラス	J1
グループ番号	19
2411516	池崎 文哉
2411540	大谷 華景
2411548	奥野 棕太
2411730	南 悠莞

1 プログラムの概要

1.1 プログラムの目的

私たちのグループでは、対戦型の 2D シューティングゲームを作成した。本作品における目的は機体进行操作し球を発射して相手に当てて相手の HP をゼロにすることである。

1.2 プログラムの主な仕様

本作品では、ゲームアプリを立ち上げた後、部屋を建てるか部屋に参加するかを選び対戦を開始することができる。対戦開始後、プレイヤーはスペースキー、Z キー、X キーによって異なる 3 種類の球を発射できる。マップ上にはランダムな位置にアイテムが生成され、取得すると回復効果や弾丸のパワーアップ効果を得られる。ゲーム終了後はリザルト画面に遷移し、そこで両者のプレイヤーがリトライを選択すると再戦することができる。

1.3 プログラムの解析

「1.2 プログラムの主な仕様」よりプログラムの実現のために必要な処理は大まかに以下のように分割できる。

- プレイヤークラスの実装
- 弾クラスの実装
- ギミッククラスの実装
- プレイヤーと弾丸やギミックの接触判定を確認する管理システム
- サーバーとクライアント間での通信の確立、切断
- 画面遷移
- ユーザーのキー入力を取得する機構

1.4 作業の分担

作業の分担として、池崎は全体のゲームの動きを統括する Model を作成した。また、画面揺れや通信の軽量化など既存のコードの改良も行った。大谷はプレイヤー、弾、ギミックなどのゲームを構成する要素の実装を行った。南は git リポジトリも管理、画面遷移やリトライ機能の実装、通信の確立、タイトル画面の設計・装飾・素材作成、SE・BGM の選定と設定を担当した。奥野はコントローラーの機能の実装や背景画像の素材の用意を主に行った。

2 設計方針

2.1 プレイヤークラスの実装

操作キャラクターはサーバー側、クライアント側の合計2つで、それぞれがプレイヤークラスのインスタンスによって管理される。よって、プレイヤークラスは座標、半径などの基本情報に加え、自身の向くべき方向とそれに伴う移動範囲の管理をする。また、どちらが勝利したかを表す勝利フラグもこのプレイヤークラスが保持する。弾の発射も、発射座標や移動方向などプレイヤーの情報に依存するものが多いため、このクラスが処理するものとする。弾を発射するときは、ゲーム管理側がプレイヤークラスの該当メソッドを呼び出す。

2.2 弾クラスの実装

キャラクターが発射する弾は1つずつそれぞれの弾クラスのインスタンスによって管理される。弾クラスは座標などのプレイヤークラスも持つ基本情報に加え、弾は相手プレイヤーに当たったときや画面外に出たときに削除する必要があるため、その弾が有効かどうか管理する。実際の削除はゲーム管理側が行い、弾クラスが行うのはその弾が有効/無効かを保持するのみである。また、爆発は弾自体の画像を切り替えることで表現し、そのために爆発の段階的な画像処理は弾クラス自体が行うものとする。

2.3 ギミッククラスの実装

ギミックの効果を実際に付与する処理はゲーム管理側で行うものとし、ギミッククラスは座標や表示時間などの基本情報と、そのギミックの種類を保持して画像を描画するのみで、具体的な効果の計算等はないものとする。ゲーム管理側が対象ギミックの種類を判別し、それに応じた効果をプレイヤーに与える。時間経過やプレイヤーが取得したことによるギミックの削除も、ゲーム管理側が行う。

2.4 プレイヤーの当たり判定の管理

ゲームにおけるプレイヤー、弾丸、ギミックの位置情報の管理は原則としてサーバー側が行う。サーバー側では両プレイヤーの位置座標が記憶されており入力をコントローラーから受け取ることによってその位置座標を逐次更新する。また弾丸及びギミックの位置は Iterator によって管理され、それらすべてに対してプレイヤーとの距離を計算することでヒット確認を行う。

2.5 接続の確立

本システムでは、ウィンドウを生成し全体を統括する役割を持つ `GameFrame` クラスが画面遷移と通信オブジェクト (`CommServer` または `CommClient`) の保持を一元管理する設計とした。通信の確立を行う段階では、UIを担当する `ServerPanel` および `ClientPanel` が通信オブジェクトを生成し、接続が完了した段階でそのオブジェクトを `GameFrame` に引き渡すことで、後のゲーム画面 (`ShootingView`) での通信を可能にしている。

通信処理には、授業 HP で配布されたサンプルコード (`Comm.java`) を利用し、TCP/IP によるソケット通信を行う。ただし、同一リンク内における通信に限定される。

2.5.1 非同期接続アルゴリズム

Comm.java と同様にサンプルコードにある tennis.java は通信を確立するためにターミナルでコマンドを実行し、サーバー側は IP アドレスの確認とポート番号の入力決定、クライアント側は接続先の IP アドレスとポート番号を入力する必要がある。この方法は特にターミナルによるコマンド実行に慣れていないユーザーにとっては操作が難しく、接続確立のハードルが高いと考えた。このため、UI 上で接続操作を行う方法に変更した。

しかし、コマンドの標準入力をただ JTextField に置き換ええるだけでは、接続確立のためのブロッキング処理が UI スレッドを占有してしまい、画面がフリーズしてしまう問題がある。これを解決するために、接続確立の処理を UI スレッドとは別のスレッドで行う非同期接続アルゴリズムを採用した。

1. ユーザーが接続操作 (ボタン押下) を行う。
2. UI スレッドとは別の Thread を新たに生成し、起動する。
3. サーバー側は標準 API のメソッドを用いて IP アドレスの確認と、ランダムなポート番号 (1024～65535) の生成を行い、クライアント側は接続先の IP アドレスとポート番号を UI 上で入力する。
4. 新スレッド内で `ServerSocket.accept()` (サーバー側) または 接続用の `Socket` (クライアント側) の生成を行う (ブロッキング処理)。
5. 接続成功後、`SwingUtilities.invokeLater` を用いて UI スレッドに割り込み、画面をゲーム画面へ切り替える。

以上のように、非同期通信確立処理を行うことで、ユーザーにとって快適な UI を提供しつつ、通信の確立を実現することができる。

2.5.2 通信におけるデータ構造

Comm の接続方式に従い、文字列ベースのプロトコルを採用している。つまり、通信の際には、送受信するデータを文字列としてシリアル化し、`send/recv` メソッドを用いてやり取りする。ただし、この際 `CommServer/Client` の区別は必須である。また、ゲーム通信時のプレイヤー座標などのデータ転送とゲーム終了後のリトライ要求/切断のメッセージを区別するため、ゲーム通信では「Data:」で始まる文字列を、リトライ要求では「Retry:」、切断は「ENd:」で始まる文字列を送受信している。

2.6 通信の実行 (ゲーム)

本ゲームはネットワーク型の対戦ゲームであるため、ゲームの状態を両プレイヤー間で共有し共通の状態にしておかなければならない。そのために、まずサーバー側でゲームの状態を完全に管理する。そしてサーバー側で計算された各物体の情報を `StringBuilder` を用いて文字列としてサーバー側からクライアント側へと伝える。また、通信の軽量化を行うために受信専用スレッドによってデータを受信し、得られた情報が上書きされて失われないようにキューに入れてそこから取り出して情報を読み取るようにする。

画面遷移は、CardLayout を用いて実装している。これにより、タイトル画面、サーバー待機画面、クライアント接続画面、ゲーム画面、結果表示画面の5つのパネルを切り替えることができる。ただし、結果表示画面のみはオブザーバーパターンを用いての終了通知と引数での勝敗のフラグの受け渡しを行うため、他と同様にコンストラクタ内でのパネルの生成ができないため、CardLayout とは別の方法で遷移を実装している。

2.9 クラス

[illegible]

3 プログラムの説明

【MoveManager クラス】

5

- `server` 自身が持っている `comm` オブジェクトがサーバーであるかクライアントであるかについての `boolean` フラグ
- `court.size_x, court.size_y` 対戦画面でのコート大きさ
- `player1, player2` プレイヤー。 `player1` は画面左側、 `player2` は画面右側の機体を表す。
- `winner` 勝者がどちらのプレイヤーかについて。初期値では空文字列が与えられている。
- `iaRunning` `MoveManager` が動いているのかのフラグ
- `msgQueue` ゲームデータについての文字列を保存するキュー

抜粋した主なメソッドは次の通り。

- `MoveManager(int x, int y, int offset, boolean server, Object comm)`: `Movemanager` のコンストラクタ。プレイヤー、弾丸、ギミックなどの各変数を初期化する。また、ゲームデータの受信を行いキューに保存させるスレッド `recvThread` を起動する。
- `update()`: メインとなるループ。データの送信および `msgQueue` に保存されたメッセージを取り出して解読する。また、サーバーの場合はプレイヤーや弾丸、ギミックの動作およびヒット確認について計算する。各動作は同クラス内に定義された別メソッドにて行われる。
- `serverLogic()`: プレイヤー、弾丸、ギミックの動作について計算するメソッド。ギミックの位置は乱数を用いてゲームフィールド内のランダムな位置に生成されるようにする。弾丸、ギミックの情報はそれぞれ `Iterator` によって管理される。 `Iterator` 内部のアイテムすべてについて、非アクティブあるいは画面外に出たなら削除し、アクティブなものについては `isHit` 関数あるいは `isHit.Gimmick` 関数によってヒット判定を計算する。弾丸が当たった場合には弾丸に備わっているメソッドを操作することで爆発アニメーションを開始させる。ギミックの場合も同様にプレイヤーがギミックに接触したらギミックの種類に応じた効果が生じるようにする。ギミックによるパワーアップの持続時間及びギミックアイテムの表示時間の管理もこのメソッド内で行う。どちらかの体力がゼロになったら `winner` に勝者を入れ `gameEnd` 関数を実行する。

```

// 爆発
if (b.getStateExplosion() != 0) { // 爆発状態なら
    if (b.explosion()) { // 爆発が進んだら
        b.setStateExplosion(b.getStateExplosion() + 1); // 状態を進める
        b.setAnimationFrames(0); // アニメーションフレームをリセット
    } else { // 爆発が進んでいなければ
        // アニメーションフレームを進める(これらの数字の判定はBulletクラス内で行う)
        b.setAnimationFrames(b.getAnimationFrames() + 1);
    }
}
// 非アクティブなら削除
if (!b.getIsActive()) {
    it.remove();
    continue;
}
boolean hit = false; // 当たったか(初期値はfalse)
// Player1への当たり判定
if (b.getOwner() != player1 && isHit(player1, b) && b.getStateExplosion() == 0) { // 自分の弾でなく、当たっていて、爆発状態でなければ
    GameFrame.playSE("music/damaged.wav", 0.5f); // SE再生
    player1.hit(damage: 1); // 1ダメージ
    hit = true; // 当たったことを記録
}
// Player2への当たり判定
else if (b.getOwner() != player2 && isHit(player2, b) && b.getStateExplosion() == 0) { // 自分の弾でなく、当たっていて、爆発状態でなければ
    GameFrame.playSE("music/damaged.wav", 0.5f); // SE再生
    player2.hit(damage: 1); // 1ダメージ kage1213, 先週 • 担当部分にコメント追加
    hit = true; // 当たったことを記録
}
}
if (hit) { // 当たっていたら
    b.setStateExplosion(1); // 爆発状態を1(爆発初期)にする
    b.setAnimationFrames(0); // アニメーションフレームをリセット
}
}

```

図 2: serverLogic() の弾丸のヒット判定の部分の一部抜粋

- gameEnd(): GameFrame クラスに対して Observer を用いてゲーム終了を通知する
- isHit(Player p, Bullet b): プレイヤーとの距離を計算し弾丸とプレイヤーの大きさを元にヒット判定を計算する。
- isHit.Gimmick(Player p, Bullet b): isHit 関数と同様にしてギミックとプレイヤーのヒット判定を計算する。
- sendServerData(): StringBuilder を用いて図 3 のようにしてゲーム情報を表現できる文字列を作成する。append() によってゲーム情報を付加していく。各情報の間には区切り文字として" " や" ", "#などを付加していくことで読み取り時にデータを分割できるようにしておく。

```
// 2. 弾情報
for (Bullet b : bullets) {
    // color取得(円で描画する場合)
    int rgb = (b.getColor() != null) ? b.getColor().getRGB() : Color.RED.getRGB();

    // 弾の型判定
    int type = 0; // 通常の弾
    if (b instanceof CurveBullet) { // 曲線弾
        type = 1;
    } else if (b instanceof UpDiagonalBullet) { // 斜め上弾
        type = 2;
    } else if (b instanceof DownDiagonalBullet) { // 斜め下弾
        type = 3;
    }
    sb.append(b.getX()).append(",").append(b.getY()).append(",")
        .append(b.getRadius()).append(",").append(rgb).append(",")
        .append(b.getShootdir()).append(",")
        .append(type).append(",")
        .append(b.getStateExplosion()).append(",")
        .append(b.getAnimationFrames()).append(";"); // ;で弾同士を区切る
}
sb.append(str: "#");
```

図 3: sendServerData における文字付加の一例

- perseServerData (String msg) : サーバ側から送られてくるデータはプレイヤー、弾丸、ギミックの三種類についてでありこの三つの区切り文字は"#"なので図 4 のようにデータの文字列を msg.split で分割し、さらにその先のデータも各形式に合わせて分割して配列の形式で格納する。また winner の中身が空文字でないならば中身の文字列に合わせて勝者を確定させ gameEnd 関数を呼び出す。

```
// # で分割
String[] sections = msg.split(regex: "#", -1);
if (sections.length < 3)
    return;
int old_hp = player2.getHp();
// 1. プレイヤー情報
// split(",", -1) にして、末尾の空文字(winner)を消さないようにする
String[] basic = sections[0].split(regex: ",", -1);
```

図 4: perseServerData における文字列解読の例

- sendClientInput(boolean isShooting, int bulletType): クライアント側の入力を伝える場合は必要となるデータ量の数が動的に変動しないため図 5 のように単純に文字列を+で結合して送信する。

```
// クライアントからサーバーへ入力情報を送信
public void sendClientInput(boolean isShooting, int bulletType) {
    if (!server) {
        // "x y isShooting"
        cl.send("Data:" + player2.getX() + " " + player2.getY() + " " + isShooting + " " + bulletType);
    }
}
```

図 5: sendClientInput における文字列作成、送信の方法

- draw(Graphics g): プレイヤー、すべての弾丸及びギミックの描画を実行

3.2 大谷

3.3 通常弾と特殊弾

文責：大谷

操作キャラクターが発射する 1 つの弾を表すクラスと、その子クラスについて説明する。子クラスは親クラスと同様の部分が多いため、まとめて記述する。

3.3.1 Bullet, CurveBullet, UpDiagonalBullet, DownDiagonalBullet

概要:

Bullet クラスは、弾の基本情報を管理するクラスである。弾の座標、速度、描画、移動処理、および爆発アニメーションの状態管理を行う。また、CurveBullet(曲線弾)、UpDiagonalBullet、DownDiagonalBullet(斜め弾) は、Bullet クラスを継承し、移動方法を定義した move() メソッドや弾の画像を変更したものである。

主なフィールド:

- x, y: 弾の座標
- speed: 弾の速度
- radius: 弾の半径
- img: 弾の画像
- state_explosion: 弾の爆発状態。通常時は 0 で、5 まで段階的に変化する。
- AnimationFrames: 爆発中において、その画像が表示されて何フレーム目か。

主なメソッド:

- Bullet(Player owner, int x, int y, float speed, int radius, int damage, int Shootdir, Color color): speed: サーバー用のコンストラクタ。所有者、座標、速度、半径、ダメージ、向きなどのフィールド値を設定する。
- Bullet(int x, int y, int radius, Color color, int Shootdir, int state_explosion, int AnimationFrames): クライアント用のコンストラクタ。受け取った state_explosion の値によって、radius と img に代入する画像を決定する。

- `move()`: 弾の移動を定義している. `Bullet` では x 方向への直線的な移動, `CurveBullet` ではそれに加えて y 方向への `Math.sin` を用いた移動, `Up(Down)DiagonalBullet` では y 方向への直線的な移動を処理している.
- `explosion()`: 弾の爆発による画像, 半径変更を処理. `state_explosion` と `AnimationFrames` の値によって画像を決定. それぞれの段階の画像は `AnimationFrames` が 0~4 の間表示され, それ以降は次の段階にうつり, `AnimationFrames` は 0 にもどる. 爆発が進行したかを `MoveManager` クラスで認識するため, 次のように `if` 文, `else if` 文の中身が実行されたときのみ `true` を返し, そうでないときは `false` を返す.

```

1      public boolean explosion() {
2          if (this.state_explosion == 1) {
3              this.img = new ImageIcon(getClass().getResource("/images/BulletExplosion1.png")).getImage();
4              this.radius = 20;
5              return true; //進行したため, true を返す.
6          } else if (this.state_explosion == 2 && this.AnimationFrames == 5) {
7              //(中略)
8          }
9          return false; //進行していないため, false を返す.
10     }

```

実装のポイント:

それぞれの継承クラスの `move()` メソッドは, 次の通り `Bullet` クラスのものをオーバーライドしたものであり, 同様の作業で容易に新たな特殊弾を実装することができる (例は `CurveBullet`).

```

1      public void move() {
2          super.move(); //Bullet の move() 呼び出し

3          //追加の動き
4          // sin の引数更新
5          time += 0.5;
6          // y 座標更新 (爆発状態でなければ)
7          if (super.getStateExplosion() == 0) {
8              int newY = (int) (y + Math.sin(time * frequency) * amplitude);
9              super.setY(newY); // y 座標セット
10         }
11     }

```

3.4 操作キャラクター

文責: 大谷

1 人の操作キャラクターを表すクラスについて説明する.

3.4.1 Player

概要:

プレイヤーキャラクターの操作, 情報 (体力, パワーアップ状態など) の管理, および描画を管理するクラス. 弾の生成処理も行う.

主なフィールド:

- `hp`, `max_hp`: 現在の体力, 最大体力

- x, y: プレイヤーの座標
- speed: プレイヤーの速度
- radius: プレイヤーの当たり半径
- img: プレイヤーの現在の画像
- biggerbullet: ギミック効果時の弾の半径増加量. 通常時は 0 に設定されている.
- statePowerup: ギミック効果を受け始めて何フレームか. 通常時は 0.

主なメソッド:

- Player(int hp, int max_hp, int x, int y, float speed, int bounds_x_min, int bounds_x_max, int bounds_y, int radius, int Shootdir, int biggerbullet, int statePowerup): コンストラクタ. 体力, 座標, 速度, 境界, 半径, 向き, パワーアップ関係のフィールド値を設定する.
- setImg(): プレイヤーの画像を決定する. statePowerup が 0 ならば通常の画像を設定, そうでなければギミック効果時用の画像を設定する. 次のような (statePowerup / 10) % 2 の値による分岐によって, 効果中 2 つの画像をフレームによって切り替える.
- moveUp(), moveDown(), moveLeft(), moveRight(): 操作キャラクターの移動を定義している.
- hit(int damage): 弾に当たったときのダメージ, 画面揺れを処理する.
- heal(int amount): 回復ギミックをとったときの回復を処理する.
- tryShoot(int type): 弾の生成処理をする.SE の再生も行う.

```

1      public void setImg() {
2          if (Shootdir == 1 && statePowerup == 0) { // サーバー側, 通常時
3              this.img = new ImageIcon(getClass().getResource("/images/player1T.png")).getImage();
4          } else if (Shootdir == 1 && statePowerup > 0) { // サーバー側, 弾半径増加時
5              if ((statePowerup / 10) % 2 == 0) { // フレームによる分岐で効果中 2 つの画像を切り替える
6                  this.img = new ImageIcon(getClass().getResource("/images/player1PowerupT1.png")).getImage();
7              } else {
8                  this.img = new ImageIcon(getClass().getResource("/images/player1PowerupT2.png")).getImage();
9              }
10         } else if (Shootdir != 1 && statePowerup == 0) {
11             //(中略)
12         }
13     }

```

実装のポイント:

弾を生成するメソッド tryShoot を Player クラスが持っているため, ギミック (後述) によって弾の情報を変化させたいとき, 「弾が当たったときにダメージを与えることと同様に Player のフィールドを変化させ, そのフィールドは tryShoot 内にあるため生成される弾が変化する」というように, 簡単に効果を付与することができるようにした. 今回の実装では Player がフィールド biggerbullet をっており, tryShoot ではそれを次のように Bullet のコンストラクタの radius の基本値に足すことで弾の半径を大きくしている.

```

1      public ArrayList<Bullet> tryShoot(int type) {
2          // 発射する弾のリスト (呼び出し側で, 元からある配列と結合する)
3          ArrayList<Bullet> newBullets = new ArrayList<>();
4          if (type == 0) {
5              // 直進弾
6              GameFrame.playSE("music/Gun_Shot.wav", 0.5f);

```

```

7          //radius の基本値に biggerbullet を足す. 通常時は0 のため, 効果時だけ変化する.
8          newBullets.add(new Bullet(this, this.getX(), this.getY(), 4,
9              15 + this.getBiggerbullet(), 1, Shootdir, null));
19      } else if (type == 1) {
11          //(中略)
12      }
13      return newBullets;
14  }

```

3.5 ギミック

文責：大谷

特殊効果や回復を付与するギミックを表すクラスについて説明する.

3.5.1 Gimmick

概要:

ゲーム中に出現するギミックを管理するクラスである. 出現座標, 効果の種類 (回復または弾強化), および出現してからの時間を管理する.

主なフィールド:

- x, y: ギミックの座標
- radius: ギミックの当たり半径
- img: ギミックの画像
- time: このギミックが出現してからのフレーム.
- type: ギミックの種類. 0 が弾半径拡大, 1 が回復.

主なメソッド:

- Gimmick(int x, int y, int radius, Color color, int time, int type): コンストラクタ. 座標, 半径, 時間, 種類などのフィールド値を設定する.
- draw(Graphics g): img の画像を現在の座標に描画する.

実装のポイント:

このクラスが行っているのは, 座標などの情報とギミックの種類の管理, そして描画だけである. 非常にシンプルな構造で保守性が高い.

3.6 南

3.7 通信確立とリトライ

文責：南

本節では, 2.5 節で説明した通信確立の流れに基づいて, 各クラスの実装内容を説明する. なお, 先述のとおり通信の基盤となる Comm.java については, 授業で配布されたひな形コードを利用している. ただし, IO コネクションが得られない場合などに強制終了しないように一部の例外処理は変更している.

3.7.1 ServerPanel

概要:

サーバー (ホスト) としてゲームを開始する際の待機画面を提供するクラスである。自身のローカル IP アドレスと乱数によって決定されたポート番号を表示し、クライアントからの接続を待機する。

主なフィールド:

- `CustomButton closeBtn`: タイトルへ戻るためのボタン。
- `String displayIP`: 自身のローカル IP アドレスを保持し、画面描画に使用する。
- `String displayPort`: 生成されたポート番号を保持し、画面描画に使用する。
- `boolean isWaiting`: 多重起動を防ぐためのフラグ。

主なメソッド:

- `serverWait()`: 接続待機処理の核となるメソッドである。`ServerPanel` のコンストラクタ内で実行される。いざ実行されると、ランダムなポート番号を生成し、別スレッドを生成して接続待機処理を行う。
- `actionPerformed(ActionEvent e)`: ボタン押下時のイベントハンドラ。`closeBtn` が押された場合、タイトル画面へ遷移する。

実装のポイント:

接続待機時のブロッキング処理が UI スレッドを占有しないよう、`serverWait()` メソッド内で新たなスレッドを生成して接続待機処理を行う。`SwingUtilities.invokeLater()` を用いることで、ゲームの開始処理を待ちリストの最後に登録し、接続成功後に UI スレッドでゲーム画面への遷移を行う設計とした。

また、一度ゲームを終了した後にもう一度サーバー待機画面に遷移した場合、ただコンストラクタで `serverWait()` を呼び出すだけでは、以前のソケットを再利用してしまい、接続時にエラーが発生してしまうため、`componentShown` イベントをトリガーにして `serverWait()` を呼び出す設計とした。これにより、サーバー待機画面が表示されるたびに新しいソケットで接続待機処理が行われるようになった。

3.7.2 ClientPanel

概要:

クライアントとしてゲームに参加するための入力画面を提供するクラスである。サーバーから IP アドレスとポート番号を教えてもらい、これをフィールドに入力することで接続を試行する。

主なフィールド:

- `CustomButton closeBtn`: タイトルへ戻るためのボタン。
- `CustomButton connectBtn`: 接続を試行するためのボタン。
- `TextField ipField`: 接続先の IP アドレスを入力するフィールド。初期値は `localhost`。
- `TextField idField`: 接続先のポート番号を入力するフィールド。

- `JLabel statusLbl`: 接続状態を表示するラベル。初期値は「NOT CONNECTED」。接続試行中は「CONNECTING...」、接続成功時は「CONNECTED!」、接続失敗時は「CONNECTION FAILED」と表示する。

主なメソッド:

- `connectToServer(String ip, int port)`: 指定された接続先へ接続要求を行う。
- `actionPerformed(ActionEvent e)`: ボタン押下時のイベントハンドラ。`closeBtn`が押された場合、タイトル画面へ遷移する。`connectBtn`が押された場合、`connectToServer()`を呼び出して接続を試行する。

実装のポイント:

`ServerPanel`と同様に、接続試行中のタイムアウト待ちなどでUIが固まらないよう、通信処理を別スレッド化している。

3.7.3 ResultPanel

概要:

ゲームの勝敗を表示し、ユーザーに次のアクション(リトライまたは終了)を選択させるクラスである。

主なフィールド:

- `CustomButton retryButton`: ゲームのリトライ要求を行う。後述の`GameFrame.tryRetry()`を呼び出す。
- `CustomButton quitButton`: ゲームの終了通知を行う。通信相手に終了通知を送信し、タイトル画面へ遷移する。

主なメソッド:

- `actionPerformed(ActionEvent e)`: ボタン押下時のイベントハンドラ。`retryButton`が押された場合、リトライ要求の処理を開始する。`quitButton`が押された場合、終了通知を送信してタイトル画面へ遷移する。

実装のポイント:

リトライボタンが押された際、即座に画面を切り替えるのではなく、ボタンを無効化をして連打を防ぎつつ、通信相手の応答を待つ。相手の応答が「Retry:RETRY」であった場合のみゲームを再生成し、それ以外の場合は相手が終了したと見做してタイトル画面へ遷移する設計とした。

3.7.4 GameFrame

概要:

アプリケーションのメインウィンドウであり、画面遷移(`CardLayout`)と通信オブジェクトの保持を一元管理するクラスである。

主なフィールド:

- `Object commSV`: `CommServer` または `CommClient` を保持するフィールド。どちらのクラスも共通の親クラスを持たない(ひな形コードの仕様)ため、`Object`型として保持し、使用時に型判定を行っている。

- `ShootingView view`: ゲームの描画を担当するクラスのインスタンス。ゲーム開始時に生成される。
- `MoveManager mm`: ゲームのデータ全般の管理を担当するクラスのインスタンス。ゲーム開始時に生成される。

主なメソッド:

- `startGame(boolean isServer, Object comm)`: 通信オブジェクトを受け取り、自身がサーバーかクライアントかを判定し、それに応じてゲームロジック (`MoveManager`) と描画 (`ShootingView`) を初期化してゲームを開始する。
- `tryRetry()`: ゲーム終了後のリトライ処理における通信同期を行う。自身と相手の両方がリトライ要求をしていた場合のみ、`retryGame()` を呼び出してゲームを再生成する。それ以外の場合は、相手が終了したと見做してタイトル画面へ遷移する。
- `retryGame()`: ゲームの再生成を行う。今あるコンポーネントを全て削除し、`startGame()` を呼び出してゲームを初期化する。
- `backToStart()`: タイトル画面へ遷移する。`CommServer/Client` を閉じ、今あるコンポーネントを全て削除し、タイトル画面のパネルを追加して `CardLayout` を切り替える。
- `update(Observable o, Object arg)`: ゲーム終了の通知を受け取るためのオーバーライドメソッド。`MoveManager` から通知を受け取ると、`ResultPanel` を生成してゲーム終了の画面へ遷移する。

実装のポイント:

ゲーム開始時にキー入力をゲーム画面に集中させるため、`startGame()` 内で `SwingUtilities.invokeLater` により待ちリストの最後に登録したうえで `requestFocusInWindow()` を呼び出している。

また、リトライ処理の通信同期を行う `tryRetry()` も、通信待ちによる UI のフリーズを回避するために別スレッド化している。そして、自分がリトライを要求しても、相手もリトライ要求をしていない時は大方コネクションが切断されているので、この際は例外処理としてテンプレートの `Comm.java` を書き換えて通常の終了通知の時と同様「QUIT」とメッセージを送信して、「相手がゲームを終了しました。」と、ポップアップで表示するようにしている。

3.7.5 Comm.java

概要:

本授業のひな形コードとして配布された通信用クラスファイルである。サーバー用の `CommServer` クラスとクライアント用の `CommClient` クラスが含まれている。

フィールド等の列挙はサンプルコードであるため割愛する。

3.8 画面遷移 (GameFrame)

文責：南

本節では、2.7 節で説明した画面遷移の実装に関して説明する。

主なフィールド:

- `JPanel mainPanel`: `CardLayout` を適用するためのメインパネル。タイトル画面、サーバー待機画面、クライアント接続画面、ゲーム画面をこのパネルに追加し、切り替える。(リザルト画面は2.5.1 節で説明した理由により、別の方法で遷移を実装している。)

主なメソッド:

- `showCard(String key)`: `CardLayout` を用いて、引数で指定されたキーに対応するパネルを表示する。

3.9 タイトル画面の装飾

文責：南

本節では、タイトル画面を主として私が作成した装飾クラスについて説明する。

3.9.1 StartPanel

概要:

ゲーム起動直後に表示されるタイトル画面を構築するクラスである。押下によるサーバー作成 (CREATE)、クライアント接続 (JOIN) 各画面への遷移と、終了 (EXIT) の管理する。

主なフィールド:

- `BackGroundPanel bgPanel`: 背景画像と星のアニメーションを描画するパネル。
- `CustomButton serverBtn, clientBtn, exitBtn`: デザインが適用された操作ボタン。

実装のポイント:

後述の SE 設定やホバー時アニメーション等のメソッドをまとめた `Design` インターフェース内の `flashImage(boolean)` メソッドと `setSound(String)` メソッドを用いて、ボタンのホバー状態に応じたアニメーションを実装している。また、後述のそれぞれのボタンのクラス (`CustomButton`) の `MouseListener` を用いて、ホバー状態を検知している。

3.9.2 BackGroundPanel

概要:

メインとボタンのマウスホバー時の2つの背景画像を読み込み、`isFlashed` フラグの状態に応じて描画する画像を切り替える機能を持つクラスである。また、背景画像の上に奥野制作の流れる星のアニメーションも同時に描画するために背景画像の透過も行っている。

主なフィールド:

- `Image bgImage_original, bgImage_flashed`: 通常時とホバー時の背景画像を保持するフィールド。
- `boolean isFlashed`: ホバー状態を保持するフラグ。true のときは `bgImage_flashed` を描画し、false のときは `bgImage_original` を描画する。

主なメソッド:

- `flashImage(boolean isFlashed)`: 引数で受け取ったホバー状態を `isFlashed` フィールドにセットし、ホバー時は星の流れる速度を遅くする演出も行う。
- `paintComponent(Graphics g)`: `isFlashed` フィールドの状態に応じて、通常時とホバー時の背景画像を切り替えて描画する。また、背景画像を透過させて星のアニメーションも同時に描画する。

実装のポイント:

`flashImage(boolean isFlashed)` メソッドをオーバーライドメソッドにすることで、`Design` インターフェースを実装するクラスであれば、どのクラスからでもホバー状態の変更とそれに伴う演出の変更を行えるようにしている。

また、背景画像の切り替えに関して、ホバー時には背景画像中のビームを光らせてまるで画像の中のスペースシャトルが発射するかのような演出を行った。これは画像編集ソフトを用いて元画像のハイライト値を上げることによって白を基本とする明るい色を強調することによって実現した。

この手法の発案当初は元画像の RGB 値をプログラム内で読み込み、ホバー時に値を緩やかに調整することでより自然なアニメーションを実装することを目標としていたが、この場合、都度新しい画像を生成する必要があり、パフォーマンス面で問題があると考え、あらかじめホバー時の画像を用意しておく方法に変更した。

3.9.3 CustomButton

概要:

タイトル画面の操作ボタンに適用するデザインを実装するクラスである。

主なフィールド:

- `Consumer<Boolean> actionBox`: そのクラス既存の `actionPerformed` に `mouseEntered` と `mouseExited` の処理を追加するためのフィールド。
- `Color lineColor, shadowColor`: ホバー状態に応じて変化する線の色と影の色を保持するフィールド。
- `boolean isHovered`: ホバー状態を保持するフラグ。true のときはホバー状態、false のときは非ホバー状態である。

主なメソッド:

- `paintComponent(Graphics g)`: `Graphics2D` クラスを用いて、小数点以下の値に対応した正確な形をしたボタンを描くためのメソッド。コンストラクタで引数として受け取った `String text` と `Font font` のとおりにボタンのテキストを中央に描く。
- `setSound(GameFrame f)`: 後述の SE 設定のためにインターフェースのメソッドをオーバーライドするためのメソッド。引数で受け取った `GameFrame` のインスタンスを用いて、ホバー状態の変更に伴う SE の再生を行う。
- `setAction(Consumer<Boolean> actionBox)`: `setSound()` と同様に、後述のホバー状態の変更に伴うアニメーションのためにインターフェースのメソッドをオーバーライドするためのメソッド。引数で受け取った `Consumer<Boolean>` を用いて、ホバー状態の変更に伴うアニメーションの処理の追加を行う。

実装のポイント:

ボタンのホバーの可否がわかるように、マウスホバーさせるとボタンとその影の色が反転するようになっている。`getWidth()` と `getHeight()` を用いて描画範囲を取得しているが、パスの太さや影とのズレ (offset) により、本体の描画可能範囲が取得範囲と異なるため、ボタンの形を描く前に実際的な描画可能範囲を再計算している。同様の理由で、テキストの描画位置も再計算している。

3.9.4 FontLoader

概要:

引数に外部からダウンロードしたフォントのファイルパスとフォントサイズを受け取るだけでフォントの設定ができるクラスである。

メソッド:

- `loadFont(String path, float size)`: 引数で受け取ったファイルパスからフォントを読み込み、引数で受け取ったサイズに設定して返すメソッド。ファイルの読み込みに失敗した場合は、デフォルトのフォントを返す。

3.10 BGM・SE 設定 (GameFrame)

文責：南

本節では、wave 形式の BGM・SE の設定のためのフィールド、メソッドを説明する。

フィールド:

- `Clip clip`: 音声の制御を行うためのフィールド。BGM と SE の両方で使用する。

メソッド:

- `playBGM(String path, float valume)`: 引数で受け取ったファイルパスから音声を読み込み、再生するメソッド。BGM なので、ループ再生する。
- `stopBGM()`: BGM の再生を停止するメソッド。
- `playSE(String path, valume)`: 引数で受け取ったファイルパスから音声を読み込み、再生するメソッド。SE なので、ループ再生せず、再生が終わったら自動的に閉じる。
- `stopSE()`: SE の再生を停止するメソッド。

3.11 奥野

4 実行例

4.1 ボタンのマウスホバーのアニメーションの実行例

ホバー前後のタイトル画面のスクリーンショットを以下に示す。

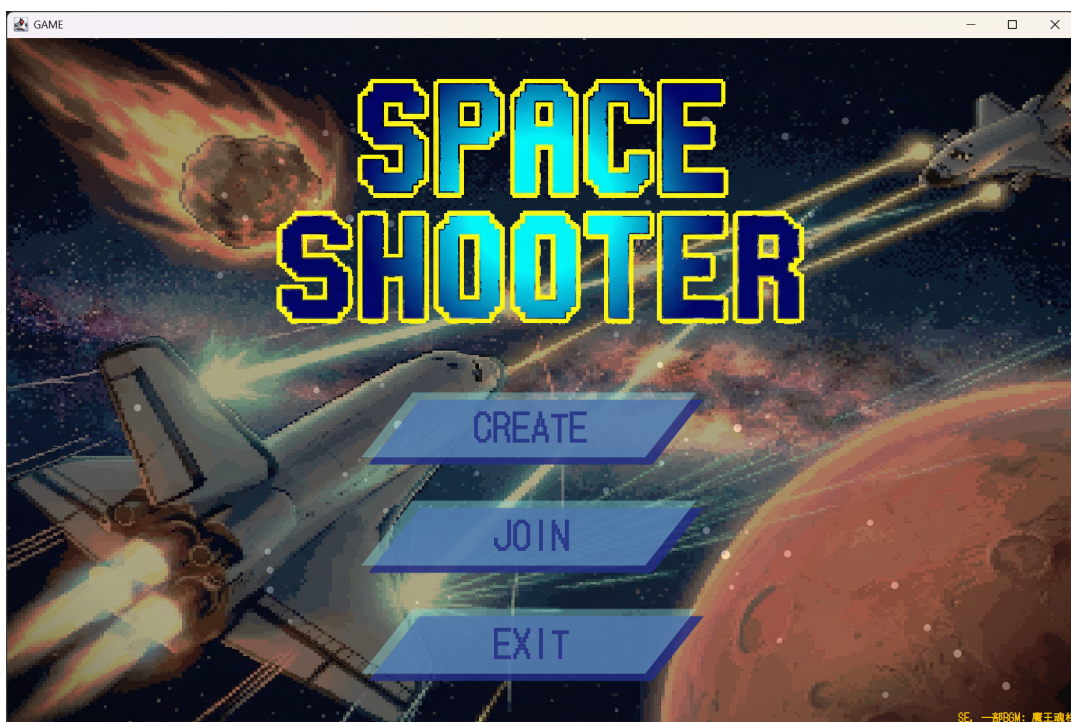


図 6: ホバー前

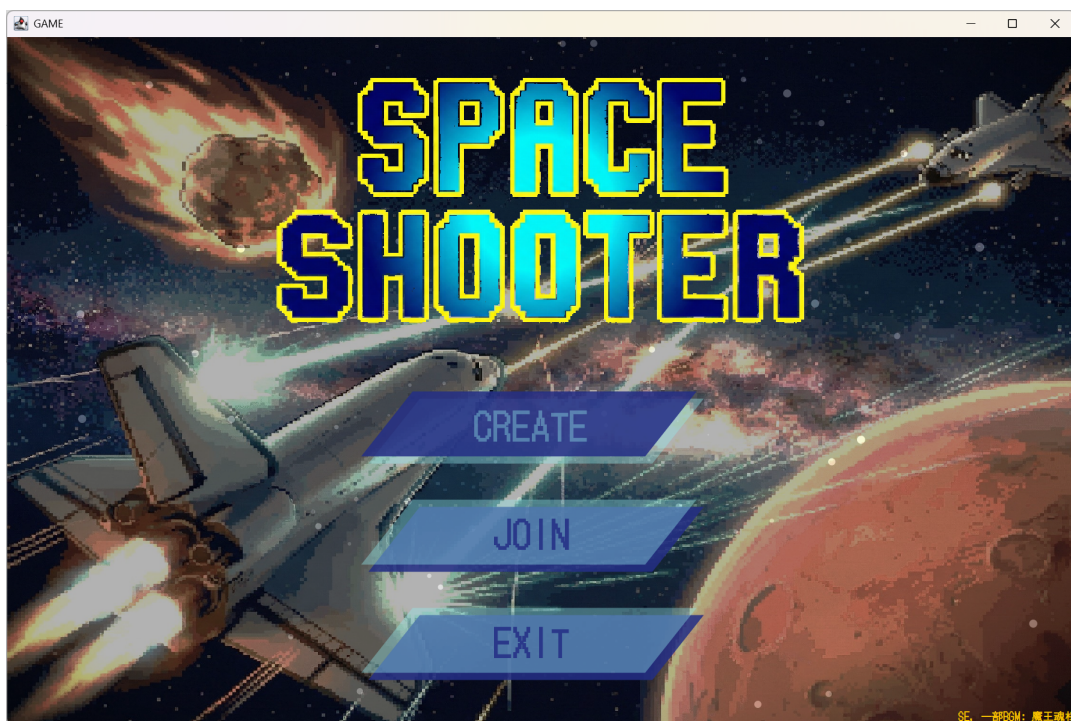


図 7: ホバー後

未定節で説明したように、ホバー前は背景画像のビームが暗く、ホバー後はビームが光っている。また、ホバー前後でボタンの色が反転している。

4.2 ゲーム終了後のリザルト画面への遷移の実行例

ゲーム終了後のリザルト画面への遷移の流れを示すスクリーンショットを以下に示す。

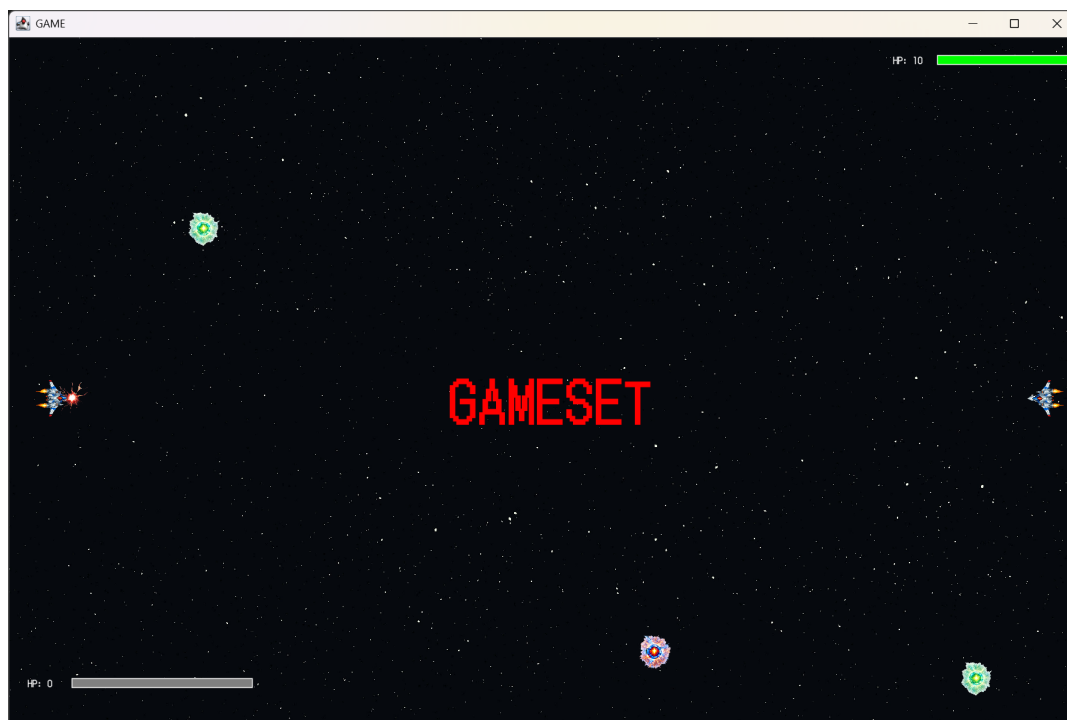


図 8: ゲーム終了直後の画面

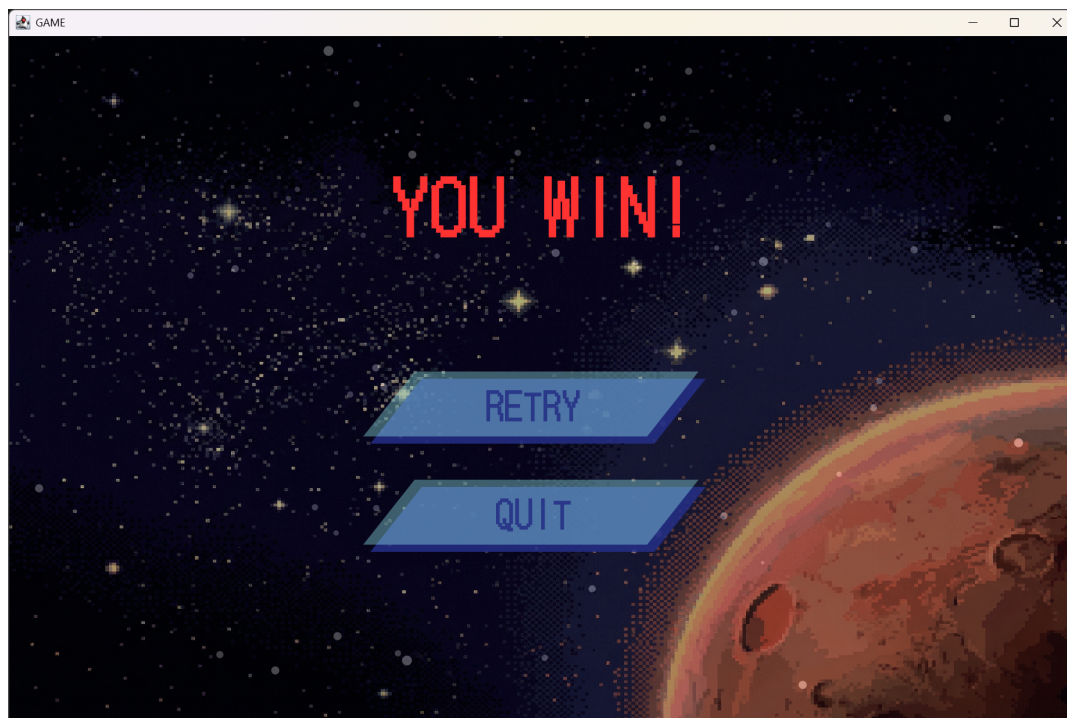


図 9: 勝者のリザルト画面

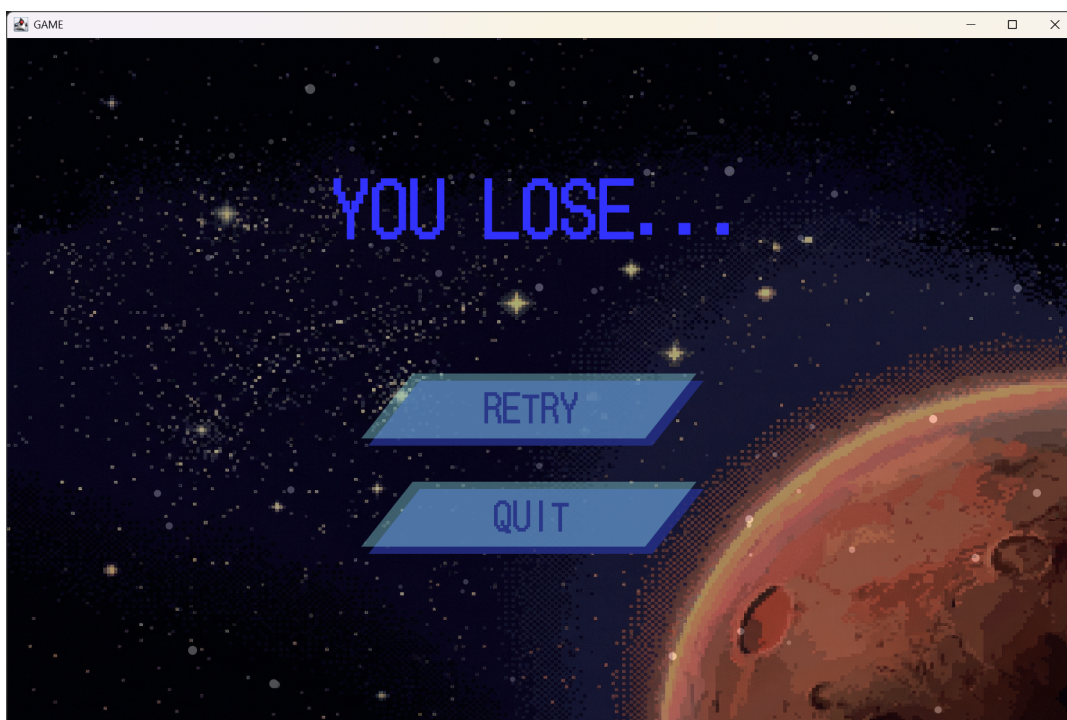


図 10: 敗者のリザルト画面

以上のように、ゲーム終了直後はキー入力拒否され、一定時間「GAMESET」と画面中央に表示された後、勝者と敗者で異なるリザルト画面へ遷移する。

4.3 リトライ要求時のコネクション切断

リトライ要求の際に片方がリトライ要求をして、もう片方が終了通知を行った場合の、リトライ要求を行った側の要求前後の画面のスクリーンショットを以下に示す。

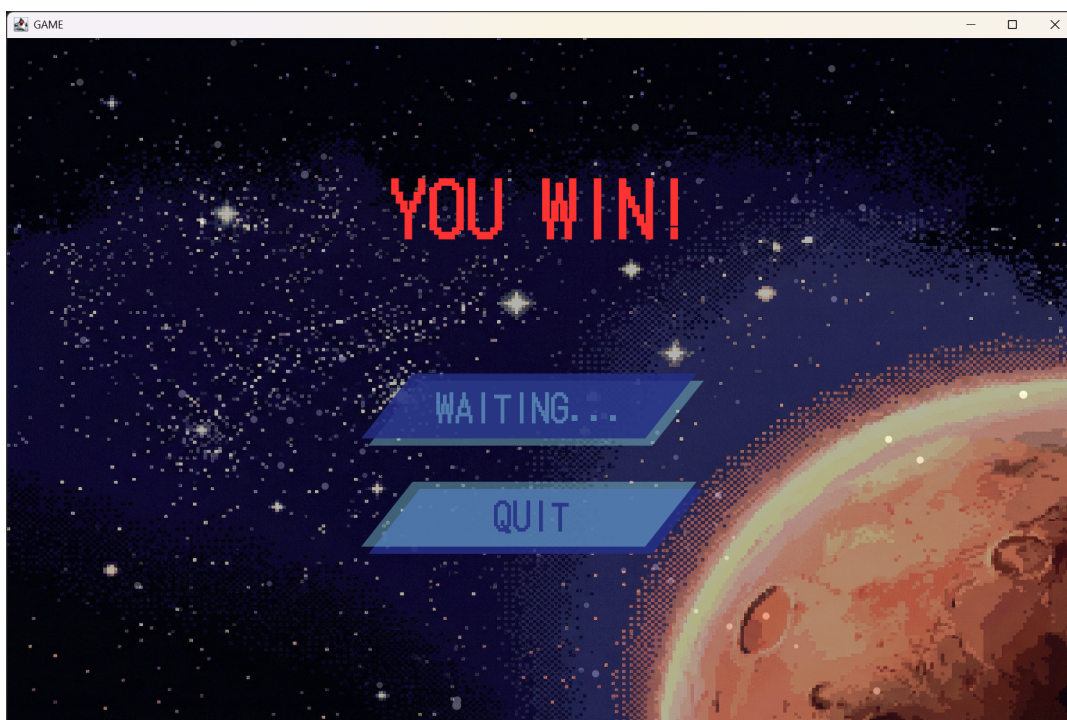


図 11: リトライ要求後の待機画面

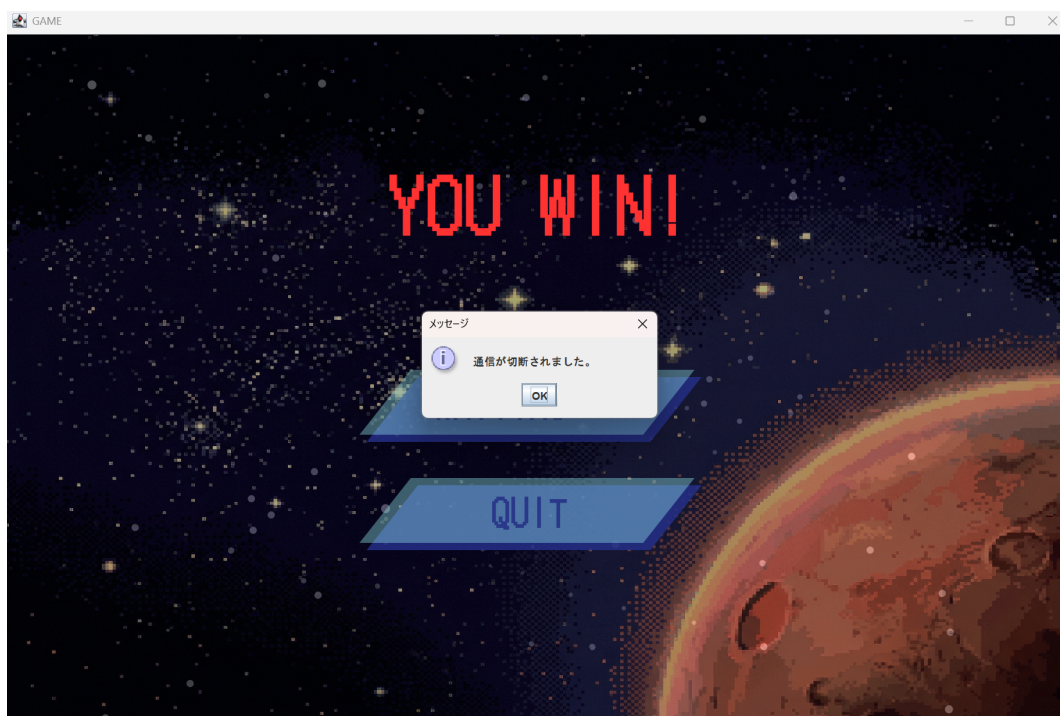


図 12: コネクションが切断後の画面

以上のように、相手からのリトライ要求の応答がない場合は、相手がゲームを終了したと見做して、ポップアップを表示している。

4.4 弾射出時の実行例

サーバー側でスペースキーを押し、通常弾の射出を行った画面のスクリーンショットを以下に示す。



図 13: 通常弾の射出時の画面 (ズーム)

所有者の座標から、所有者の向いている方向に向かって弾が射出されている。

4.5 弾爆発の実行例

クライアント側の曲線弾がサーバー側に当たったときの爆発のうちの 1 フレームを以下に示す。



図 14: 曲線弾の爆発時の画面 (ズーム)

曲線弾の画像が、特定の爆発画像に変化している。

4.6 ギミック取得の実行例

サーバー側における、通常時と弾半径拡大ギミック取得時のスクリーンショットを以下に示す。

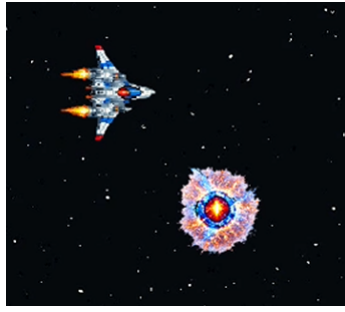


図 15: 通常時 (ズーム)

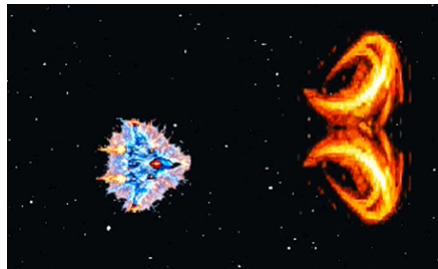


図 16: 弾半径拡大ギミック取得時 (ズーム)

操作キャラクターの画像がギミック取得時のものに变化し、弾半径が大きくなっている。

5 考察

5.1 プログラムについての考察

通信型のゲームを作成するうえで軽量化という問題は非常に重要な課題だと感じた。今回作成したゲームは現代におけるゲームの中ではかなり要素の少ないものであるが、それでもスレッドのスリープ時間やゲームデータの送信方式をある程度工夫しないとラグが発生してしまっていた。今回使用したライブラリのメソッドがゲーム用の通信としてあまり最適化されていない点もあるとはいえ、もっと大きい規模のゲームを作成するならば今回のもの以上に通信の最適化について気にしなければならないと考えられる。今回作成したゲームにおいて挙げられる改善点としては、`Comm.java` を改善することである。具体的には、今の `Comm.java` は TCP での通信であり、信頼性が高い代わりに通信速度が遅いためゲームでの通信には適さない。そのため、通信を UDP ベースで行えるようにし弾の発射や被弾などの重要なイベントのみ再送しそれ以外は古いデータを上書きするという方式にして軽量化を行うようにしたい。

5.2 ゲームシステムについての考察

今回作成したゲーム作品が多くの人に評価してもらえ、最優秀賞をとることができたのは対戦型 2D シューティングゲームというコンテンツ自体の面白さの面が大きかったと考える。2D 対戦型のシューティングゲームはゲームの構造としてシンプルながら奥深さがあるため、プレイしたいと思ってもらいやすかった。しかしながら今回作成したゲームでは、2D 対戦型シューティングゲームの土台を作ること

にかなり苦戦したため、このゲーム独自の奥深さを十分に出すことができなかった。当初の予定では、フィールドを何種類か作成しダメージギミックなどそのステージ独自のギミックを作成する予定であったのでその部分は改善点である。また、今回はデバッグの時間をあまりとれていないため本来意図していない動作をした時などでバグが発生する可能性もある。これらの点をこれから改善していきたい。

5.3 通信の確立に関する考察

今回の実装では、サーバーとクライアントを明確に区別して通信を行った。計画当初はチーム戦もできるような実装にしたいと考えていたがこの実装方法では後付けで追加出来なかった。

今後、チーム戦や観戦者モードを実装するためには、サーバーとクライアントの区別をなくし、group18が行っていたように必要な処理(データ送受信など)をインターフェースにまとめて、サーバーとクライアントの両方で共通の親クラスを作成して、必要に応じてオブジェクトを生成する方法が考えられる。

また、今回は Comm サーバーを用いた関係上、TCP 通信を行った。もし、ローカル対戦だけでなく、インターネット対戦も行えるようにする場合、group18 のようにターン性のゲームであれば TCP 通信でもよいが、今回のようなリアルタイム性のゲームで TCP を用いると通信の遅延が致命的になる可能性が高いと考えられる。このため、UDP 通信を採用することが好ましいと考えられる。UDP 通信ならコネクションの確立/切断やパケットの再送制御等の処理が行われないため、通信の遅延を減らすことができると思われるからである。

5.4 弾, ギミックの実装に関する考察

今回、弾クラスとギミッククラスは全く別のクラスとして作成した。しかし、この2つのクラスのフィールドは共通するものが多く、またプレイヤーのように入力で操作するものではない。よって、より抽象的な「ゲーム盤面上に存在する非操作オブジェクト」とした `abstract class ObjectField` のような抽象クラスを作成し、それを継承して `Bullet` クラスや `Gimmick` クラスを作成する方法もあったと考えられる。この利点として、プログラムの規模がより大きくなったとき、今回の `MoveManager` にあたるような管理ロジックにおいて、弾なのかギミックなのかは考慮せず「ゲーム盤面上に存在する非操作オブジェクト」として一括で処理したいときなどに扱いやすくなる点が挙げられる。

5.5 ギミックの拡張性に関する考察

ギミック機能は、開発終盤に短時間で作成したものであり、拡張性が考慮されていない。実際、ギミッククラスはコードの短期完成を優先したために非常に簡潔で最低限なものである。そのため、現在の仕様ではギミックの種類を増やすためには、`MoveManager` の該当箇所を拡張する必要がある。要素の追加でゲームロジックを管理するクラスを大きく書き換える必要があることは望ましくない。そのため、今後の改良点として、可能な限り効果の処理もギミッククラス内で行うものにすることが挙げられる。

6 各自の反省と感想

6.1 池崎

私たちのグループでは、コード作成の初めにクラス図を作成して分担を行うことができていなかったため並行して作業することがあまりできておらず作業の効率化が行えていなかった。一人での作成であればファイル構造が複雑化してもある程度問題なく開発できるが、グループでのプログラム作成や保守

性を考えると、もっとファイル構造の簡略化を行い可視性を上げる必要があると思う。そのためにも、オブジェクト指向についてもっと考える必要があると考える。ゲーム独自の要素としてのギミックの種類を増やすことが今後の課題である。また、リトライ機能において片方がRETRYを押しもう片方はQUITを押すなど予期しない動作を行った時の動きの正常化など技術力が足りず直しきれなかったバグもあったのでこれらも改善したい。Java 言語などのオブジェクト指向言語では MVC モデルなどでの構造の簡略化が非常に大事だと感じた。プログラミング演習の授業を通してグループでのプログラミングを行い開発の難しさやファイル構成を事前に決めることに大切さなどを学ぶことができた。

6.2 大谷

今回の演習では、開発計画がはっきりできていなかったことが反省点であると感じる。必要そうなものを作ってみて、それならばこれも必要、といった形で進めてしまった部分があった。まずはクラス群の全体像を組み立てて、そこから実際にコードを書く方がスムーズに進んだと思う。考察 5.5 で述べたようにギミックの拡張性まで手が回らなかったことも、これが理由の 1 つであると感じる。開発の人数やプログラムの規模が大きくなると立案から完成までの見通しの重要性がより高まると考えられるため、今後このような開発をするときは、今回の反省を生かした体制で取り組みたいと思う。

これまでプログラミングは演習授業の問題など、1 人で書くものしか取り組んでこなかった。グループプログラミングでは分担、データの共有の方法 (Git Hub)、人が書いたコードを自分が書いたコードと結びつけることなど、プログラミングそのものの技術以外にも多くのことを意識する必要があった。実際の開発では、むしろ 1 人で書くことがないと思われるので、今回の経験を活かしたい。

6.3 南

git の管理をしていたわけだが、経験はあっても使い慣れてはいないのでマージコンフリクトが起きた時の対処に苦労した。以降はブランチを上手く使い分けて作業を行うようにしたい。

また、発表のある週に突貫工事を行ってなんとか人前に見せれる形になったが、発表の 1 週間前にギミックの量ではなくて見た目の意匠を凝らすことに専念したのは正解だったと思う。発表のデモの時間は短いので「面白そう」と思わせるにはギミックを煩雑にしてやりこみ要素を増やすよりも、見た目のインパクトを重視した方が効果的だと考えたからである。実際、ゲーム以外のアニメーションやフォント含めたグラフィックの統一感や BGM、SE にこだわり、一般にあるゲームを思い浮かべて被ダメージの画面揺れと赤い点滅を直前に提案し実装してもらい、その他弾やエフェクトにもモデル班にこだわってもらったのは功を奏したようで、発表後の観衆のコメントを全て拝見したが見た目に関するポジティブな評価がとても多かった。これにより 1 票差ではあったが最優秀賞を獲れたので非常に良かったと思う。ただ、ゲーム開始前のカウントダウン等の比較的簡単に実装できる演出や、考察に書いた通信の課題点の解決策の実装や、ゲーム上での必殺技や妨害ギミックや回避行動など、やり残したことは多々あったので、今後は時間配分をより上手く行って、やりたいことを全て実装できるようにしたいと思う。

また、普段は個人でしかも C 言語という全く違う環境でしかプログラミングをしていないので、Java やオブジェクト指向、MVC モデルなどの学習と実行はとても大変だった。ただ、こういったアプリケーションの制作は今回の演習のような制作環境が主流だと思うので、慣れていかないと行かないと思った。

最後に、高校の頃にグループでゲームを制作したものの中途半端で終わってしまった自分としては、今回の演習はリベンジであったので、最優秀賞をとるところまで形にできたのは非常にうれしかった。今後このようなアプリケーションの制作に携わる機会があれば、今回の経験を活かして、より良いものを作れるようにしたいと思う。

付録1：操作法マニュアル (ユーザーズマニュアル)

java GameFrame をターミナル上で実行, または jar ファイルを実行することでアプリが立ち上がる。ゲームを立ち上げたら、プレイヤーのうちどちらかが CREATE ボタンを押して部屋を立てる。もう片方のプレイヤーは JOIN ボタンを押し、部屋を立てた側のプレイヤーの画面に表示されている IP アドレスと ROOM ID(ポート番号) をクライアント側の画面に入力することで通信が確立し対戦が始まる。ゲーム中、プレイヤーは矢印キーで上下左右に動くことができる。また、スペースキーを押して直進弾、Z キーで波弾、X キーで斜め弾を発射することができる。ゲーム中にはフィールド上にアイテムが生成される。図 20 のようなアイテムを拾うと、発射する弾の大きさが図 22 のように大きくなる。また、図 21 のようなアイテムを拾うと体力が回復する。どちらかの体力が 0 になればゲーム終了となりリザルト画面に遷移する。リザルト画面ではゲームを終了するか再戦するかを選ぶことができ、両方のプレイヤーが RETRY を押すことで再戦ができる。意図しない挙動を引き起こす可能性があるため、両者の選ぶボタンは異なるものにする。

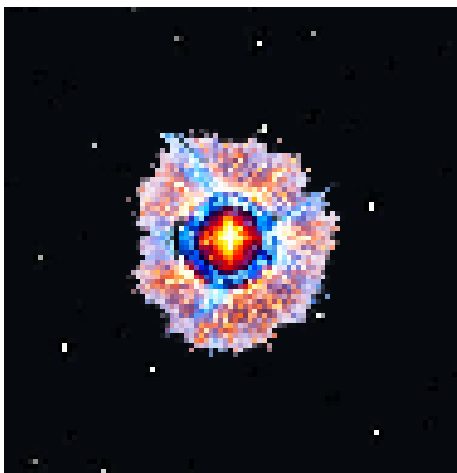


図 17: パワーアップアイテムギミック

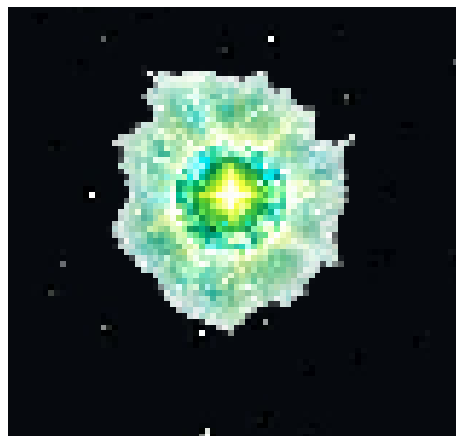


図 18: 回復アイテムギミック



図 19: パワーアップアイテム取得前後での弾の大きさの変化

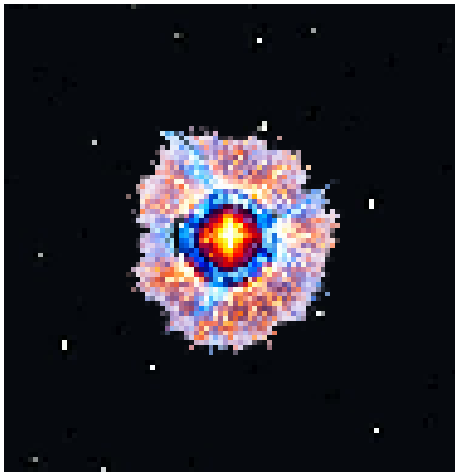


図 20: パワーアップアイテムギミック

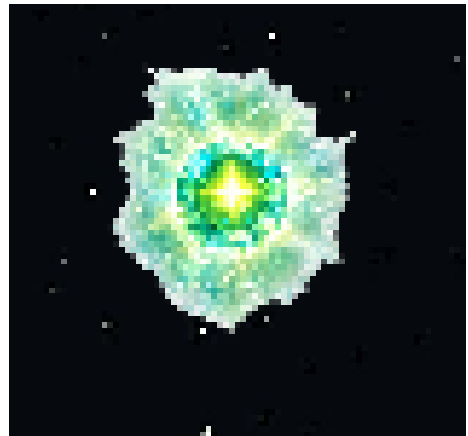


図 21: 回復アイテムギミック



図 22: パワーアップアイテム取得前後での弾の大きさの変化

付録2：プログラムリスト

プログラムリストは本文中には説明に必要な部分だけ抜粋して掲載して、プログラムリスト全体はレポートの最後に「付録」として付ける。プログラムコードには、主なメソッド、フィールドの説明など最低限のコメントは付ける。

cat -n hash.c などとして、行番号付きのプログラムリストを生成して、
`\begin{verbatim}....\end{verbatim}`を用いて記述すると読みやすい。ページ数が多くなる場合は、`\small`や`\footnotesize`を用いるとよい。

```
1 struct item* insert(struct item s){
2     int i; struct entry *p, *chain;
3     i=h(s.key);
4     chain=H[i];
5     if(chain){
6         while(chain && comparekeytype(chain->term.key, s.key)) chain=chain->next;
7         if(chain) return &(amp;chain->term); /* 同じ鍵が見つかった 場合*/
8     }
```